YP-RAKSHA

Real-time AI Knowledge Screening & Hazard Audit

Full Technical Documentation

Field	Value
Product name used in this document	YP-RAKSHA
Name in the running software	Gateway (default). Brand mark: Raksha.
Subtitle in the product	Real-time AI Knowledge Screening & Hazard Audit
Organisation default	YesPanchi Group of Companies
Document version	Repository baseline, Guard agent 1.1.20
Document date	12 October 2026
Classification	Internal
Audience	Developers, architects, AI engineers, security specialists, integrators, and auditors
Source of truth	This Git repository. Not a certification claim.

The repository default product string is Gateway, set by GATEWAY_PRODUCT_NAME. The brand string defaults to Raksha. This document set uses the requested product name YP-RAKSHA for the same implementation.

This document describes behaviour found in source code, configuration samples, and existing Markdown guides. Where a feature is partial or absent, the text says so. It does not claim ISO certification, complete hazard detection, or guaranteed AI safety.

Contents

Contents 2
1. Executive summary 4
2. Scope, objectives, requirements, and assumptions 4
2.1 Scope 4
2.2 Objectives visible in the implementation 4
2.3 Assumptions 4
3. Technology stack and repository structure 5
3.1 Principal directories 5
4. System architecture and data flow 6
4.1 Component diagram (logical) 6
4.2 Trust boundaries 6
5. Module analysis 7
5.1 HTTP gateway 7
5.2 Configuration store 7
5.3 Skills repository 7
5.4 Browser Guard 7
5.5 User interface 7
6. AI and knowledge-screening behaviour 8
6.1 What is implemented 8
6.2 What is not implemented 8
7. Access control 8
7.1 Roles 8
7.2 Directory provisioning 9
8. APIs 9
9. Data 9
10. Security, privacy, and standards mapping 10
11. Testing evidence 10
12. Build and deployment (technical view) 10
13. Gaps, limitations, and recommendations 11
14. Handler and route map 11
14.1 Inference path 12
15. Plugins 12

16. MCP 12
17. Skills serving detail 12
18. Browser AI data flow 13
19. Persistence tables 13
20. Authentication mechanics 13
21. Threat model notes 13
22. Configuration identity 14
23. CI, quality, and dependency practice 15
24. Glossary 15
25. Inference API catalogue 15
26. Two different screening mechanisms 16
    26.1 Gateway plugin named guardrails 16
    26.2 Browser Guard rules 16
27. Browser AI API catalogue 17
28. RBAC resource map 18
29. Plugin catalogue 19
30. Session and password behaviour 19
31. Sequence: browser prompt 20
32. Sequence: gateway chat 20
33. Data lifecycle 20
34. Settings pages 20
35. Standards alignment appendix 21
36. Known implementation gaps 21
37. Appendix: repository map for reviewers 22

If the contents list is empty, press Ctrl+A then F9 in Word, or right-click the contents and choose Update Field.

1. Executive summary

YP-RAKSHA, as implemented in this repository, is an enterprise AI gateway with an endpoint browser-control agent. One Go service accepts OpenAI-compatible and provider-native API traffic, applies governance, and stores configuration and logs. A React workspace is the administration console. A Python desktop agent, Gateway Guard, inspects selected public AI websites in the employee browser and can block, redact, or warn.

The product does not ship a single trained hazard-classification model that scores every prompt. Hazard handling is policy driven. Gateway traffic can pass through optional guardrail plugins. Browser traffic is matched to administrator rules that are either regular expressions or an optional AI Guard Bot that calls a configured model. Requests that never enter the gateway or the Guard proxy are not screened.

Two planes share one database and one dashboard:

- Gateway plane: unified inference API, providers, virtual keys, budgets, routing, MCP, prompts, skills, plugins, observability, and RBAC.
- Browser AI plane: target websites, guard rules, local or network MITM proxy, prompt logs, search logs, and fleet agents.

2. Scope, objectives, requirements, and assumptions

2.1 Scope

- In scope: code under core, framework, transports, plugins, ui, apps/browser-guard, deploy, and docs.
- In scope: authentication, RBAC, SCIM, on-premises Active Directory LDAP sync, virtual keys, and audit logs.
- Out of scope for automatic screening: traffic that does not use the gateway API and is not steered through Guard PAC or the network proxy.
- Out of scope: a claim that every hazard category is detected. Detection is limited to configured rules and optional model checks.

2.2 Objectives visible in the implementation

- Give applications one API surface while keys for many model vendors stay on the server.
- Limit spend and rate by virtual key, user, team, and customer budgets.
- Record LLM and MCP activity for operators.
- Let administrators decide which public AI sites employees may use and what prompt content is blocked, redacted, or warned.
- Provision people from cloud directories (SCIM) or on-premises Active Directory.

2.3 Assumptions

- PostgreSQL is the intended production database. SQLite exists for local development paths.

- A reverse proxy terminates public TLS. The Go process default listen port used in local docs is 6000.
- Provider credentials are stored as configured keys, not hard-coded in source that this review treats as production secrets.
- Human administrators own rule quality. False positives and false negatives are expected for regex and model rules.

3. Technology stack and repository structure

Layer	Implementation
Gateway language	Go. HTTP engine: fasthttp. Router: fasthttp/router.
UI	React 19, Vite, TypeScript. Workspace routes under ui/app/workspace.
Desktop Guard	Python agent in apps/browser-guard/agent. Packaged with PyInstaller. Windows EXE and macOS .app.
Inspection proxy	mitmproxy addon apps/browser-guard/proxy/browser_ai_proxy.py.
ORM / config	GORM in framework/configstore.
Logs	framework/logstore, including Browser AI tables.
Containers	docker-compose.yml and deploy/docker.
Charts	deploy/helm.

3.1 Principal directories

Path	Role
core/	Request schemas, provider adapters, routing helpers.
core/providers/	Vendor adapters (OpenAI, Anthropic, Azure, Bedrock, Gemini, Vertex, Mistral, Cohere, Groq, Ollama, and others).
framework/	Config store, log store, encrypt, feature flags, RBAC helpers, object store.
transports/gateway-http/	HTTP handlers, session, SCIM, Active Directory, Browser AI downloads, embedded UI.
plugins/	Guardrails, governance, logging, telemetry, semantic cache, prompts, OTel, connectors, and others.
ui/	Administration workspace.

apps/browser-guard/	Agent, proxy, installer, and release packages.
docs/	Existing Markdown guides and this generated set.

Provider packages present under core/providers include openai, azure, anthropic, bedrock, bedrockmantle, gemini, vertex, mistral, cohere, groq, cerebras, deepseek, fireworks, huggingface, nebius, ollama, openrouter, perplexity, replicate, runware, runway, sgl, vllm, xai, elevenlabs, parasail, and opencode. The exact call surface of each adapter is the Go files in that folder. Not every adapter implements every modality.

4. System architecture and data flow

4.1 Component diagram (logical)

```
Employee browser
  -> PAC (local 127.0.0.1:18195 or server PAC)
  -> Guard MITM 127.0.0.1:18103
  -> rule engine (regex or AI Guard Bot)
  -> POST /api/browser-ai/intercept
  -> PostgreSQL logs + Workspace Browser AI

Application or IDE
  -> Gateway /v1 or provider-native path
  -> auth (virtual key or session)
  -> optional plugins (guardrails, cache, governance)
  -> provider adapter
  -> upstream model API
  -> logs, budgets, response
```

4.2 Trust boundaries

- Boundary A: public internet and upstream model vendors. Provider API keys must not be given to end users.
- Boundary B: employee laptop. The Guard trusts its local proxy and a company CA installed for TLS inspection. Only domains listed as targets are intended to be intercepted.
- Boundary C: gateway process. Session cookies and virtual keys are the caller identity.
- Boundary D: database. Configuration, users, rules, and logs.
- Boundary E: directory. SCIM bearer token or LDAP bind account. Those credentials can create users.

Implemented control: gateway admin sessions bypass per-route RBAC in transports/gateway-http/handlers/rbac_enforce.go when session.Role is admin. The user role is resolved from the stored role permission catalogue. Sub-admin API rights come from the sub_admin role seed and later edits to that role, not from each person's checkbox matrix alone. Page visibility for user and sub_admin uses allowed_sections.

5. Module analysis

5.1 HTTP gateway

Handlers live in transports/gateway-http/handlers. Session login, user CRUD, and password policy are in session.go. RBAC enforcement is in rbac_enforce.go and framework/rbac/enforce.go. PathRequirementFor maps API prefixes to a resource and operation. Unknown routes are not all gated. A nil requirement means the route is not RBAC-gated.

5.2 Configuration store

framework/configstore persists users, teams, customers, virtual keys, roles, prompts, skills, and workspace settings through GORM. Users are table governance_users (framework/configstore/tables/user.go). Username is unique. Email is indexed and is not a unique constraint. Role, status, budget, rate limit, allowed prompt repositories, and allowed_sections are columns on the user.

5.3 Skills repository

Skills are stored and served by transports/gateway-http/handlers/skills.go and skills_serving.go. Public marketplace URLs do not use the dashboard session. Claude Code fetches marketplace JSON and clones plugin git URLs. Codex clones the marketplace git URL. Cursor and Antigravity harness names are included in framework/configstore/skills.go SkillHarnessNames so git routes are registered. Serving is an in-process git smart HTTP implementation. It does not require a system git binary.

5.4 Browser Guard

apps/browser-guard/agent/agent_config.py sets AGENT_VERSION_BAKED. At the time of this document the baked value is 1.1.20. Runtime version prefers VERSION.txt beside the install. Local proxy port used in operations notes is 18103. Local PAC port is 18195. Release artefacts under apps/browser-guard/release include Gateway_Guard.exe, Gateway_Guard_Setup.exe, Gateway_Guard_Windows.zip, Gateway_Guard_macOS.zip, and Gateway_Guard_Setup.pkg. The pkg file on disk at review time was older than the 1.1.20 zip and is not what the macOS update command installs. The update command downloads the macOS zip.

5.5 User interface

Workspace navigation is defined in ui/components/sidebar.tsx and mirrored by grant keys in ui/lib/constants/workspaceSections.ts. Sections include Observability, Browser AI, Models, MCP Gateway, Plugins, Governance, Guardrails, Adaptive Routing, Prompt Repository, Skills Repository, and Settings. Child grants use keys such as observability/llm-logs.

6. AI and knowledge-screening behaviour

6.1 What is implemented

- Gateway inference is pass-through to the selected provider after auth, budgets, and optional plugins. There is no universal classifier on every /v1 request unless a guardrail plugin is configured and ordered to run.
- Browser Guard extracts prompts, uploads, and search queries for hosts that match Target Websites, then evaluates Guard Rules.
- Rule actions implemented in the product language are BLOCK, REDACT, and WARN.
- A rule may be a pattern or an AI Guard Bot that calls a model the administrator selects. The bot is optional per rule. It is not a fixed embedded model.
- Prompt logs and search logs are stored and shown under Browser AI.

6.2 What is not implemented

- No evidence of a single accuracy benchmark, published false-positive rate, or false-negative rate in this repository.
- No evidence that every hazard class (for example all regulated data types) is detected unless an administrator wrote a rule for it.
- AI Guard Bot quality depends on the chosen model, the rule prompt, and the text the proxy extracted. Extraction can miss content the site renders in a way the addon does not parse.

Human oversight is required. Administrators should review prompt logs, tune rules, and treat model judgements as advisory enforcement, not as a guarantee.

7. Access control

7.1 Roles

Role	Login behaviour found in code
admin	API middleware allows the request without the resource catalogue. Workspace sections are not filtered.
user	Pages come from allowed_sections. If none are stored, the UI and API fall back to Prompt Repository. Write APIs depend on the user role permission rows. The seed grants View and Read, not Create, Update, or Delete.
sub_admin	Pages come from allowed_sections. API operations come from the sub_admin role catalogue. The seed is full control for some resources (Logs, Virtual Keys, Prompt Repository) and view or absent for others. Selecting a page on the user form does not by itself grant every write API for that page.

The Roles and Permissions screen can show a per-person matrix derived from allowed_sections: full ticks for admin, all operations on mapped resources for sub_admin, and View or Read only for user. Saving that person updates allowed_sections. The enforcement function ResolvePermissions loads the role row, not a separate per-user operation list. Treat the global All Users and All Sub Admins policies as the live API policy.

7.2 Directory provisioning

Cloud SCIM is implemented at /scim/v2 for Users and Groups. Providers accepted in configuration are okta, entra, and keycloak. The bearer token is required. New SCIM users are created with role user. Matching uses external id, email, or user name, so the same email is not inserted twice. Groups become teams. A configured default team can be applied.

On-premises Microsoft Active Directory uses LDAP in transports/gateway-http/handlers/active_directory.go. Sync creates approved users with role from default_role, which defaults to user. It skips an existing username (case-insensitive). It does not look up email, so the same mailbox with a different username can create a second account. It assigns the SCIM default team if one is configured, sends a temporary password mail, and asks the prompt lifecycle to create a prompt named for the email. It does not set allowed_sections, a virtual key, or an admin role.

8. APIs

Inference routes follow OpenAI-compatible /v1 paths and native integration prefixes. Exact route tables are registered in the transport server. Browser AI routes include /api/browser-ai/logs, targets, rules, agents, setup downloads, and intercept. Governance routes live under /api/governance and /api/session. SCIM routes are unauthenticated except for the bearer token middleware.

Errors use HTTP status codes. SCIM errors use the SCIM error schema with status, detail, and optional scimType. Dashboard RBAC failures return 403 with an insufficient-permissions or section message.

Do not publish real tokens. Examples in the system manual use placeholders such as <VIRTUAL_KEY> and <SCIM_BEARER_TOKEN>.

9. Data

PostgreSQL is configured with DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, and DB_SSL_MODE. Migrations run from the config store. Browser AI logs, agents, rules, and targets are part of the log store and related tables. Skill files may live in object storage or database blobs. Prompt bodies live in the prompt tables.

Personal data includes usernames, emails, prompt text stored in Browser AI logs, and virtual-key usage. Logs can contain confidential prompts. Retention and access should be treated as an operational control. This repository does not, by itself, prove a fixed retention period for every log type.

10. Security, privacy, and standards mapping

The following mapping is an engineering aid. It is not a certificate and not evidence of a completed audit.

Theme	Where the code supports it	Gap
Access control (ISO 27001 A.5 / 27002 access)	Session login, RBAC resources, allowed_sections, virtual keys.	Per-user operation matrix is not the runtime policy.
Cryptography	Password hashes, TLS to AD when ldaps or StartTLS is used, Guard CA for inspection.	AD can be configured with insecure TLS skip. That is an explicit risk.
Logging	LLM logs, MCP logs, audit logs, Browser AI prompt and search logs.	Log content may itself be sensitive.
Supplier AI services	Provider adapters. Keys stay server-side.	Upstream vendors process prompts. That is outside this codebase.
AI system lifecycle (ISO 42001 themes)	Configurable policies, human-administered rules, logs for review.	No formal model-risk file, no measured accuracy, no claim of continuous monitoring of model drift.

11. Testing evidence

Go tests exist beside handlers, including browser AI, RBAC scope grants, and SCIM Entra cases (scim_entra_test.go). Python tests exist under apps/browser-guard/agent/tests. This document does not quote pass rates. A test file proves intent and some regressions. It does not prove production accuracy of hazard detection.

12. Build and deployment (technical view)

- Gateway image: deploy/docker/Dockerfile.local referenced by docker-compose.yml service gateway_tech.
- Required environment includes APP_PORT, SERVER_DOMAIN, database settings, ADMIN_EMAIL, ADMIN_PASSWORD, and GATEWAY_GUARD_SECRET.
- Optional network proxy: docker compose profile network-proxy.
- UI development server port in Vite defaults from UI_PORT or 3100. It proxies to the gateway.
- Guard Windows build is a Windows plus Inno Setup pipeline. macOS .app and pkg require a Mac build for a new native binary. Windows can refresh VERSION.txt and encrypted code into an existing .app, which is what the 1.1.20 macOS zip reflects.

13. Gaps, limitations, and recommendations

- Align sub-admin API rights with the features selected on the user form, or change the screen so it does not imply full access for every selected feature.
- Check email as well as username during Active Directory sync.
- Persist per-user operation overrides if the matrix is meant to change live access.
- Rebuild Gateway_Guard_Setup.pkg on a Mac when a signed package at 1.1.20 is required. The zip used by the macOS updater is the current channel.
- Publish an operator review cadence for Guard rules and prompt logs.
- Keep provider keys and the Guard shared secret out of tickets and documents.

14. Handler and route map

Route registration is split by handler. The files below are the places to start a code review. A route existing in a RegisterRoutes function does not mean every deployment has enabled the feature behind it.

Handler file	Responsibility
handlers/inference.go	Chat and text completions presented to callers.
integrations/router.go	Provider-native integration routes.
handlers/providers.go	Provider configuration API.
handlers/governance.go	Virtual keys, users, teams, customers, budgets.
handlers/session.go	Login, logout, password change, session cookie.
handlers/rbac_enforce.go	Which API prefix requires which resource.
handlers/workspace.go	Workspace settings, SCIM, Active Directory routes.
handlers/scim_handlers.go	SCIM Users and Groups.
handlers/active_directory.go	LDAP test and user sync.
handlers/skills.go and skills_serving.go	Skill CRUD and public git marketplaces.
handlers/mcp.go and mcp*.go	MCP catalog, sessions, OAuth, inference.
handlers/plugins.go	Plugin install and ordering.
handlers/guardrails.go	Guardrail rule configuration.
handlers/logging.go	Log queries.
handlers/health.go	Health endpoint.
handlers/ui.go	Embedded React application.
handlers/browser AI download handlers	Setup zip, version header, intercept ingest.

14.1 Inference path

1. The caller sends an OpenAI-style request with a virtual key or another accepted credential.
2. The gateway resolves the virtual key, checks status, budgets, and rate limits.
3. Routing rules, complexity routing, or an explicit model name select a provider.
4. Plugins that are enabled and ordered for that request may modify or reject it. A guardrail plugin is not active unless an administrator configured it.
5. The provider adapter in core/providers translates the request and calls the vendor.
6. Usage is written to logs and budget counters. The response is returned to the caller.

Streaming responses use the same auth and budget checks before the stream starts. A mid-stream vendor failure is a provider error, not a hazard classification.

15. Plugins

The plugins directory contains optional pipeline stages. Names present in the tree include governance, guardrails, logging, telemetry, semantic cache, prompts, otel, and data connectors. Each plugin runs only when it is enabled in configuration. Document a deployment by listing the enabled plugins from the Plugins screen, not by assuming the whole directory is active.

16. MCP

MCP Gateway pages cover a catalog, a library, tool groups, auth sessions, OAuth grants, and settings. Virtual keys can carry MCP configuration. OAuth issuance and consent handlers exist so a client can be authorised without pasting a long-lived vendor token into the laptop. Tool calls that pass through the gateway can be logged as MCP logs. Tools that a user runs outside the gateway are not visible.

17. Skills serving detail

Harness	How the client is expected to connect
Claude Code	HTTP marketplace JSON, then clone of plugin URLs.
Codex	Clone of the marketplace git URL itself.
Cursor	Clone of /api/skills/serve/cursor. The marketplace file is .cursor-plugin/marketplace.json and plugin sources are paths inside that clone, not remote URLs.
Antigravity	Plugin manifest at the repository root with name and description only. Install command uses the gateway-all-skills plugin URL.

Public skill git routes do not require a dashboard session. Treat published skill text as visible to anyone who can reach the gateway URL. Do not put secrets inside a skill file.

18. Browser AI data flow

7. An administrator lists target hosts and rules.
8. The laptop agent sets a local PAC. Default local ports used by the product are 18195 for PAC and 18103 for the inspection proxy.
9. Only matching hosts are intended to be decrypted and inspected.
10. The proxy extracts prompt text, uploads, or search queries when its parsers recognise the site.
11. Rules run. Actions are block, redact, or warn.
12. The event is posted to the gateway intercept API and appears in Prompt Logs or Search Logs.
13. An optional network proxy container exists behind the compose profile network-proxy. It is off by default. Employee production path documented in compose comments is the laptop program.

GATEWAY_FAIL_OPEN is set to 0 on the optional network proxy, meaning a failure to reach the gateway is not configured there as a silent allow. Confirm the laptop agent policy in the installed config before you state the same for desktops.

19. Persistence tables

GORM models under framework/configstore/tables include, among others, users, teams, business units, customers, virtual keys and their provider and MCP bindings, providers, skills and skill file blobs, prompts and prompt sessions, RBAC roles, access profiles, audit logs, SMTP, login lockout, password reset, circuit breaker policies, feature flags, model pricing, and workspace settings. Browser AI log tables live with the log store. Exact column lists are the Go structs. Do not invent extra columns in an integration.

20. Authentication mechanics

- Dashboard users authenticate with username or email and a password. Passwords are stored as hashes.
- A temporary password from user create, SCIM, or AD sync forces a password change. The policy requires length and character classes implemented in the session handler.
- Login lockout and device tables exist. Use them as evidence that lockout is a designed control. Confirm the live thresholds in configuration rather than assuming a number.
- API callers use virtual keys. A virtual key is not a user session and does not open the admin UI by itself.
- SCIM uses a bearer token. AD sync uses a bind account stored in the AD configuration.

21. Threat model notes

Threat	Relevant control	Residual risk
Stolen vendor API key	Keys stored on the gateway. Users receive virtual keys.	An admin who can open providers can read or replace keys. Protect that role.
Stolen virtual key	Budgets, rate limits, status, and	A key can still spend until it is

	audit of usage.	revoked.
Employee pastes secrets into an unlisted AI site	No control. The site is not a target.	Policy and training. The product will not see the paste.
Regex misses a secret format	Add or tune a rule. Optional model rule.	False negatives remain.
Model rule blocks harmless text	Warn action instead of block. Human review of logs.	False positives remain.
AD insecure TLS	Prefer ldaps and a trusted certificate.	Skip-verify is available and is unsafe on a hostile network.
SCIM token leak	Rotate the token. Restrict network path to the IdP.	Anyone with the token can create users while SCIM is enabled.
Skill repository is public	Do not publish secrets in skills.	The marketplace URL is intentionally unauthenticated.

22. Configuration identity

docker-compose.yml passes these identity variables into the container. Defaults are shown so operators can see what an empty override produces.

Variable	Compose default
GATEWAY_PRODUCT_NAME	Gateway
GATEWAY_BRAND_NAME	Gateway
GATEWAY_PRODUCT_SUBTITLE	Real-time AI Knowledge Screening & Hazard Audit
GATEWAY_COMPANY_NAME	YesPanchi Group of Companies
GATEWAY_COMPANY_SHORT_NAME	YesPanchi
GATEWAY_FOOTER_SUBTITLE	Gateway - Real-time AI Knowledge Screening & Hazard Audit
APP_HOST	0.0.0.0
LOG_LEVEL	info
LOG_STYLE	json
DB_TYPE	postgres
DB_SSL_MODE	disable
GATEWAY_GUARD_REQUIRE_SECRET	1
GATEWAY_PAC_ALLOW_QUERY_PROXY	0

APP_PORT, SERVER_DOMAIN, database credentials, ADMIN_EMAIL, ADMIN_PASSWORD, GATEWAY_GUARD_SECRET, and CONTAINER_NAME have no safe default. Compose fails closed if they are missing from .env.

23. CI, quality, and dependency practice

Go tests and Python agent tests are in the tree. This review did not execute a full suite and does not report a pass percentage. Dependency updates should follow your normal process: review go.mod and ui/package.json, run tests, and avoid committing .env. There is no statement in code that a vulnerability scanner runs on every commit. If your pipeline has one, document that pipeline separately.

24. Glossary

Term	Meaning in this product
Virtual key	A gateway credential with budgets and provider scope. End users should use this, not a vendor master key.
Allowed sections	Comma-separated workspace page grants stored on the user.
Guard rule	An administrator policy applied to intercepted browser AI traffic.
Target website	A host the Guard is instructed to inspect.
SCIM	Directory provisioning API at /scim/v2.
AI Guard Bot	Optional model call used by a guard rule. Not a built-in certified classifier.

25. Inference API catalogue

These routes are registered in transports/gateway-http/handlers/inference.go. A route being registered means the gateway accepts the call. The selected provider adapter must implement that modality or the call fails at the vendor.

Method and path	Purpose
GET /v1/models	List models visible to the caller.
POST /v1/chat/completions	Chat completion.
POST /v1/completions	Text completion.
POST /v1/responses	Responses API. Also retrieve, delete, cancel, and input items.

POST /v1/embeddings	Embeddings.
POST /v1/rerank	Rerank.
POST /v1/ocr	OCR.
POST /v1/audio/speech	Speech synthesis.
POST /v1/audio/transcriptions	Transcription.
POST /v1/images/generations	Image generation. Edits and variations are separate routes.
POST /v1/videos	Video generation, plus list, retrieve, content, delete, and remix.
POST /v1/batches	Batch create, list, retrieve, cancel, and results.
POST /v1/files	File upload, list, retrieve, delete, and content.
POST /v1/containers	Container and container-file routes.
POST /v1/responses/input_tokens	Token count.
POST /v1/responses/compact	Compaction.

Callers send Authorization: Bearer <VIRTUAL_KEY>. Request bodies follow the OpenAI-compatible shapes those handlers decode. This document does not paste a full JSON schema for every modality. Read the handler when you integrate a less common route such as video or containers.

26. Two different screening mechanisms

The repository contains two controls that both sound like guardrails. They do not run on the same traffic.

26.1 Gateway plugin named guardrails

- Code: plugins/guardrails. Plugin name constant is guardrails.
- It runs inside the gateway request pipeline only when the plugin is enabled and ordered.
- Rules can be CEL expressions. The CEL environment exposes request.model, request.prompt_id, request.virtual_key_id, and request.virtual_key_name.
- The provider switch in initializeProviders enables a regex provider. Any other provider name is skipped.
- Input and output validation methods exist on the provider interface. They apply to gateway requests, not to random websites.

26.2 Browser Guard rules

- Code: apps/browser-guard and /api/browser-ai.
- Actions in the proxy are BLOCK, REDACT, and WARN. BLOCK is preferred when several rules match.

- A rule may be a pattern or an AI Guard Bot. The bot is tested through POST /api/browser-ai/rules/test-bot. Regex can be drafted through POST /api/browser-ai/rules/generate-regex. Both are administrator tools, not a guarantee of detection quality.
- Ollama model names can be listed with GET /api/browser-ai/ollama-models when that local runtime is reachable. That does not mean Ollama is required.

A chat call to POST /v1/chat/completions is not inspected by Browser Guard. A ChatGPT page in the browser is not inspected by the gateway guardrails plugin unless that page's traffic is also sent through the gateway, which the normal website does not do.

27. Browser AI API catalogue

Registered in transports/gateway-http/handlers/browser_ai.go. Agent calls such as heartbeat and intercept are authenticated with the guard secret, not with a user session. Admin calls use the dashboard session and RBAC.

Area	Routes
Prompt logs	GET, DELETE /api/browser-ai/logs. Stats, bulk-delete, and delete by id.
Search logs	GET, POST, DELETE /api/browser-ai/search-logs, plus bulk-delete and delete by id.
Rules	GET, POST /api/browser-ai/rules. Import, update, delete, test-bot, generate-regex.
Targets	GET, POST /api/browser-ai/targets. Import, update, delete.
Controls	GET and PUT /api/browser-ai/controls.
PAC	GET /api/browser-ai/pac and /api/browser-ai/proxy.pac.
Setup	download.zip, download-windows.zip, download-mac.zip, Windows exe names Gateway_Guard_Setup.exe and Raksha_Guard_Setup.exe, rebuild, info, rebuild-history, proxy-bundle json and zip.
Agents	List, heartbeat, wait-command, pause, resume, delete, bulk-delete, remote uninstall, uninstall key, contact email.
Fleet	GET and PUT /api/browser-ai/fleet-config. GET /api/browser-ai/agents/settings.
Intercept	POST /api/browser-ai/intercept and /api/browser-ai/intercept-file. GET attachment by id.
Insights and mail	GET /api/browser-ai/insights/stats. POST /api/browser-ai/send-warning-email.

License	GET /api/browser-ai/license, GET license/server-id, POST license/activate. Licence terms are not defined in this document.

Log delete query periods used by tests are 1d, 7d, 30d, and all. An invalid period is rejected. A bad date format is rejected.

28. RBAC resource map

framework/rbac/enforce.go PathRequirementFor decides the resource. Admin sessions skip this map. If no requirement is returned, the route is not RBAC-gated. GET /api/config and GET /api/rbac/me/permissions are explicitly ungated for an authenticated session.

Path prefix	Read resource	Write note
/api/logs	Logs Read	Deletes require Logs Delete.
/api/logs/stats and histograms	Logs or Dashboard	Dashboard charts.
/api/mcp-logs	MCPLogs or Logs	Deletes allow MCPLogs or Logs.
/api/browser-ai/logs	Logs Read	Deletes require Logs Delete.
/api/browser-ai other	Logs View	Writes use Logs Update, including rules and targets.
/api/plugins	Plugins or Observability	Update on write.
/api/providers, /api/keys, /api/models	ModelProvider	Follows the same resource.
/api/governance/virtual-keys	VirtualKeys or Governance	
/api/governance/teams	Teams or Governance	
/api/governance/customers	Customers or Governance	
/api/governance/routing-rules	RoutingRules or Governance	
/api/mcp	MCPGateway	Tool groups use MCPToolGroups.
/api/guardrails	GuardrailsConfig or GuardrailsProviders	
/api/prompt-repo	PromptRepository	
/api/skills	SkillsRepository	Public git serve paths are a separate unauthenticated surface.
/api/scim configuration UI	UserProvisioning	The IdP protocol itself is /scim/v2 with a bearer token.
/api/roles and /api/rbac	RBAC	

/api/audit-logs	AuditLogs	
/api/smtp-config and proxy config	Settings	

Browser AI is enforced as the Logs resource. The workspace section key is browser-ai. A role can therefore see a mismatch if Logs is granted in the API catalogue but the sidebar section was not stored on the user, or the reverse.

29. Plugin catalogue

Each directory under plugins/ is an optional pipeline component. Presence in the tree is not the same as enabled in a deployment.

Directory	What the code is for
governance	Budgets, routing, complexity analysis, circuit breaker hooks.
guardrails	CEL rules and a regex provider on gateway traffic.
logging	Request and response logging.
telemetry	Metrics export. A Prometheus compose file is included.
otel	OpenTelemetry HTTP and gRPC export.
semanticcache	Semantic cache lookup.
prompts	Prompt resolution used by other plugins.
maxim	Maxim integration plugin.
connectors	Observability connectors such as the views for Kafka, Pub/Sub, BigQuery, Datadog, New Relic.
modelcatalogresolver	Resolves model catalog entries.
jsonparser	JSON parsing helper plugin.
mocker	Mock responses for tests and demos.
compat	Request compatibility and parameter dropping.

30. Session and password behaviour

- Login is POST /api/session/login. Missing or wrong credentials return 401 with the text Invalid username or password.
- Temporary password change is POST /api/session/change-temporary-password.
- Forgot password is POST /api/session/forgot-password. Reset is POST /api/session/reset-password.

- If PASSWORD_RESET_SECRET is missing or shorter than 32 characters, the session handler logs that OTP reset will return HTTP 500. Set a long random secret before you rely on email reset.
- Generated temporary passwords use upper case, lower case, digits, and a symbol from a limited set, and are at least 8 characters. The login policy matches that shape.
- Login lockout storage exists so one account cannot be sprayed without limit. Exact thresholds belong in the live security settings, not in a guessed number.

31. Sequence: browser prompt

```
Employee types in a watched site
  -> local PAC 127.0.0.1:18195
  -> local MITM 127.0.0.1:18103
  -> proxy extracts text
  -> rules: BLOCK stops, REDACT changes text, WARN notifies
  -> POST /api/browser-ai/intercept with guard secret
  -> prompt log row
  -> administrator opens Browser AI, Prompt Logs
```

32. Sequence: gateway chat

```
App sends POST /v1/chat/completions
  Authorization: Bearer <VIRTUAL_KEY>
  -> key status, budget, rate limit
  -> routing or explicit model
  -> enabled plugins, including guardrails only if configured
  -> core/providers/<vendor>
  -> vendor HTTPS API
  -> usage log and budget update
  -> response to the app
```

33. Data lifecycle

- Create: administrators, SCIM, or AD sync insert governance_users. Virtual keys, rules, and targets are separate records.
- Use: inference writes log rows and budget counters. Browser intercept writes prompt or search logs and may store an attachment.
- Delete: log APIs support single delete, bulk delete, and period delete (1 day, 7 days, 30 days, or all). Deleting a log is an administrator action and is itself subject to Logs Delete.
- Retain: no single retention number is hard-coded for every table. If you need a legal retention period, enforce it with an operational job and the delete APIs. Do not assume the product already does this.
- Export: treat prompt text as confidential. Do not attach raw logs to tickets.

34. Settings pages

The Settings section in ui/lib/constants/workspaceSections.ts contains Client Settings, Compatibility, Caching, Security, Proxy, API Keys, Performance Tuning, and Feature Flags. Paths sit under /workspace/config. API writes for SMTP, proxy, and vector store configuration require the Settings resource.

35. Standards alignment appendix

This appendix lists themes an auditor might ask about. It is not a statement of conformity and not a certificate.

Theme	Evidence you can show	What you cannot claim from this repo alone
Access control	Roles, allowed_sections, virtual keys, SCIM bearer, AD bind.	That every screen tick is enforced as an API permission.
Cryptography	Password hashes, optional LDAPS, Guard CA for inspection.	That TLS is enabled on the database. Compose defaults DB_SSL_MODE to disable.
Operations security	Docker restart policy, health routes, JSON logs.	That backups and a SIEM are already running.
Supplier relationships	Provider adapters. Keys stay on the server.	The vendor's own compliance.
AI policy and oversight	Human-authored rules, logs, block redact warn, optional model judge.	Measured accuracy, bias tests, or continuous model monitoring.
Privacy	Prompt logs exist and can be deleted by period.	A completed privacy impact assessment or a fixed retention schedule.

36. Known implementation gaps

14. The per-user Roles and Permissions matrix updates allowed_sections. API enforcement uses the role catalogue. Those two can disagree.
15. Sub-admin seed rights are not full control of every selected sidebar feature.
16. Browser AI API checks use the Logs resource. Sidebar grants use the browser-ai section. They are not the same key.
17. The user role still receives Prompt Repository when allowed_sections is empty.
18. Active Directory sync matches username only. The same email with a different username creates a second user. Cloud SCIM matches email and external id.
19. AD sync does not assign allowed_sections, a virtual key, or a non-user role.
20. Gateway guardrails ignore provider types other than regex.
21. CEL guardrail variables do not include the raw prompt text. A CEL rule in this plugin cannot, by itself, scan prompt content. Content scan on the gateway path depends on the regex provider. Content scan on the browser path depends on Guard rules.
22. Mac Gateway_Guard_Setup.pkg in the release folder was older than the 1.1.20 zip at review time.
23. Customer budget can appear in the prompt key panel. Hiding it from the user role was requested and is not described here as already done.

24. Public skill git endpoints do not require a dashboard login.

37. Appendix: repository map for reviewers

Question	Start here
Who may call an API?	framework/rbac/enforce.go and handlers/rbac_enforce.go
How is a user stored?	framework/configstore/tables/user.go
How does login work?	transports/gateway-http/handlers/session.go
How does SCIM work?	handlers/scim_handlers.go
How does AD sync work?	handlers/active_directory.go
How are skills served?	handlers/skills_serving.go
How does the browser proxy decide?	apps/browser-guard/proxy/gateway_proxy_parts/extract_office_backend.py
What version is the agent?	apps/browser-guard/agent/agent_config.py AGENT_VERSION_BAKED
What does compose require?	docker-compose.yml
What pages exist?	ui/lib/constants/workspaceSections.ts

End of full technical documentation.